

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №1
по дисциплине «Математическая логика и теория формальных
языков»
на тему «Лексический анализатор»
Вариант 16

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Востров А. В. _____

«____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание лексического анализатора	4
1.1 Структура транслятора	4
1.2 Лексический анализ	4
1.2.1 Назначение лексического анализа	5
1.3 Формальная модель лексического анализатора	6
1.4 Синтаксические диаграммы	6
1.5 Обработка ошибок	9
2 Особенности реализации	10
2.1 Класс TokenType	10
2.2 Класс Token	10
2.3 Класс LexicalAnalyzer	11
2.3.1 Конструктор класса	11
2.3.2 Методы класса	12
3 Результаты работы	16
Заключение	20
Список использованных источников	21

Введение

Цель работы: разработать лексический анализатор входного текста в соответствии с вариантом работы.

Вариант 16 Входной язык содержит только латиницу, логические выражения, разделенные символом ; (точка с запятой). Логические выражения состоят из идентификаторов, констант **0** и **1**, знака присваивания (**:=**), операций (**|**, **&**, **+**, **~**), знаков сравнения (**=**, **!=**, **<**, **>**) и квадратных скобок (**[]**).

Длина идентификатора ограничена 15 символами. Программа также должна допускать наличие комментариев неограниченной длины во входном тексте.

Задачи программы

1. Разбиение входного текста на лексемы.
2. Классификация лексем по типам (идентификатор, константа, оператор присваивания, оператор сравнения, разделитель и т.д.).
3. Проверка ограничений на длину имен идентификаторов.
4. Обработка однострочных и многострочных комментариев произвольной длины.
5. Вывод содержательных сообщений об ошибках, обнаруживаемых в процессе лексического анализа текста.
6. Порождение таблицы выделенных в тексте программы лексем.

1 Математическое описание лексического анализатора

1.1 Структура транслятора

Транслятор осуществляет преобразование цепочек языка в выходные данные, и процесс трансляции связывается с двумя основными этапами (см. рис. 1).

1. Блок-*распознаватель* обеспечивает лексический анализ текста и строит структуру выходной цепочки на основе порождающей грамматики входного языка.
2. Блок-*генератор* использует построенную структуру для генерации выхода, выражающего смысл входной цепочки.

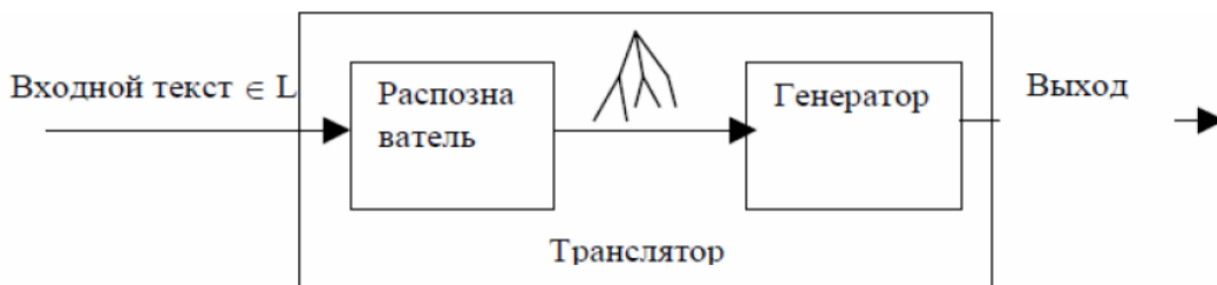


Рис. 1: Структура транслятора

В реальных трансляторах этапы распознавания и генерации разделены не так явно, но во всех случаях синтаксическая трансляция основана на построении структуры входной цепочки целиком или по частям.

1.2 Лексический анализ

В реальных трансляторах языков программирования первой фазой трансляции является *лексический анализ* входной программы.

Лексический анализ – первый проход транслятора (см. рис. 2), обеспечивающий предварительную обработку входного текста с выделением в нём структурно значимых единиц – *лексем*.

Лексема – минимальная единица языка, имеющая смысл.

- В естественных языках лексемами являются слова (словоформы).
- В языках программирования лексемами являются имена, служебные слова, константы, операторы (знак присваивания ($:=$), знаки сравнения ($==$, $!=$, $>$, $<$)) и т.д.

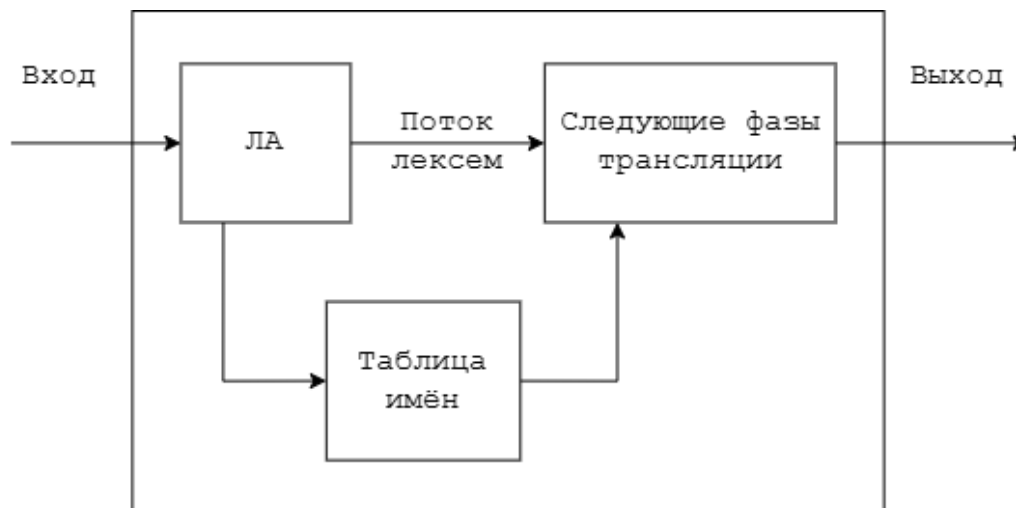


Рис. 2: Структура транслятора с лексическим анализатором

1.2.1 Назначение лексического анализа

В результате лексического анализа определяются:

- *значение лексемы*, определяющее подстроку символов входной цепочки, соответствующих распознанному классу лексемы.
- *класс лексемы*, определяющий общее назначение категории элементов, обладающих общими свойствами (идентификатор, целое число, строка символов и т.д.).

В зависимости от класса, значение лексемы может быть преобразовано во внутреннее представление уже на этапе лексического анализа. Например, числа могут быть преобразованы в двоичное машинное представление для более компактного хранения.

Устройство, осуществляющее сопоставление класса с каждым отдельным символом, называется **транслитератором**. Среди наиболее типичных классов символов можно привести:

- *буква* – класс, с которым сопоставляется множество букв (необязательно одного алфавита);
- *цифра* – множество символов, относящихся к цифрам;
- *разделитель* – пробел, перевод строки, возврат каретки;
- *игнорируемый символ* – символы, встречающиеся во входном потоке, но игнорирующиеся анализатором (например, служебный код);
- *запрещенный символ* – символы, не относящиеся к алфавиту языка, но встречающиеся во входной цепочке;
- *прочие* – символы, не вошедшие ни в одну из определенных категорий.

1.3 Формальная модель лексического анализатора

Пусть заданы:

- Σ – входной алфавит, задающий множество допустимых символов;
- T – множество типов токенов;
- D – множество допустимых значений токенов.

Тогда лексический анализатор реализует отображение

$$F_{lexer} : \Sigma^* \rightarrow (T \times D)^*,$$

где:

- Σ^* – множество всех возможных цепочек над Σ ;
- $(T \times D)$ – множество пар (тип токена, значение).

В данной работе:

- $\Sigma = \{\text{буквы латиницы}, 0, 1, :=, |, \&, +, \sim, ==, !=, <, >, [,]\}$;
- $T = \{\text{идентификатор, константа, логический оператор, оператор сравнения, оператор присваивания, скобка, разделитель}\}$;
- $D = D_{\text{ind}} \cup \{0, 1, :=, |, \&, +, \sim, ==, !=, <, >, [,]\}$, где:

$$- D_{\text{ind}} = \left\{ w \in \Sigma_{\text{id}}^+ \left| \begin{array}{l} 1 \leq |w| \leq 15, \\ w[1] \in \{A, B, \dots, Z, a, b, \dots, z\}, \\ \forall i \in [2, |w|] : w[i] \in \Sigma_{\text{id}} \end{array} \right. \right\}, \text{ где}$$
$$\Sigma_{\text{id}} = \{A, \dots, Z, a, \dots, z, 0, \dots, 9\}.$$

1.4 Синтаксические диаграммы

Синтаксические диаграммы реализуют графическое представление правил формирования лексем. На рис. 3 приведена диаграмма работы лексического анализатора, соответствующего варианту работы.

Обозначения на диаграмме:

- y1 – символ в буфер;
- y2 – выдать идентификатор и его номер;
- y3 – выдать константу 0;
- y4 – выдать константу 1;
- y5 – выдать лексему "|";
- y6 – выдать лексему "&"

y7 - выдать лексему "+";
y8 - выдать лексему "~";
y9 - выдать лексему "==" ;
y10 - выдать лексему "!=" ;
y11 - выдать лексему "<"
y12 - выдать лексему ">" ;
y13 - выдать лексему ":"=
y14 - выдать лексему "["
y15 - выдать лексему "]"
y16 - выдать лексему ";" .

Лексемы в данном языке программирования: идентификаторы на латинице, константы 0 и 1, логические операторы (|, &, +,), операторы сравнения (==, !=, <, >), знак присваивания (:=), левая квадратная скобка, правая квадратная скобка, разделитель (;).

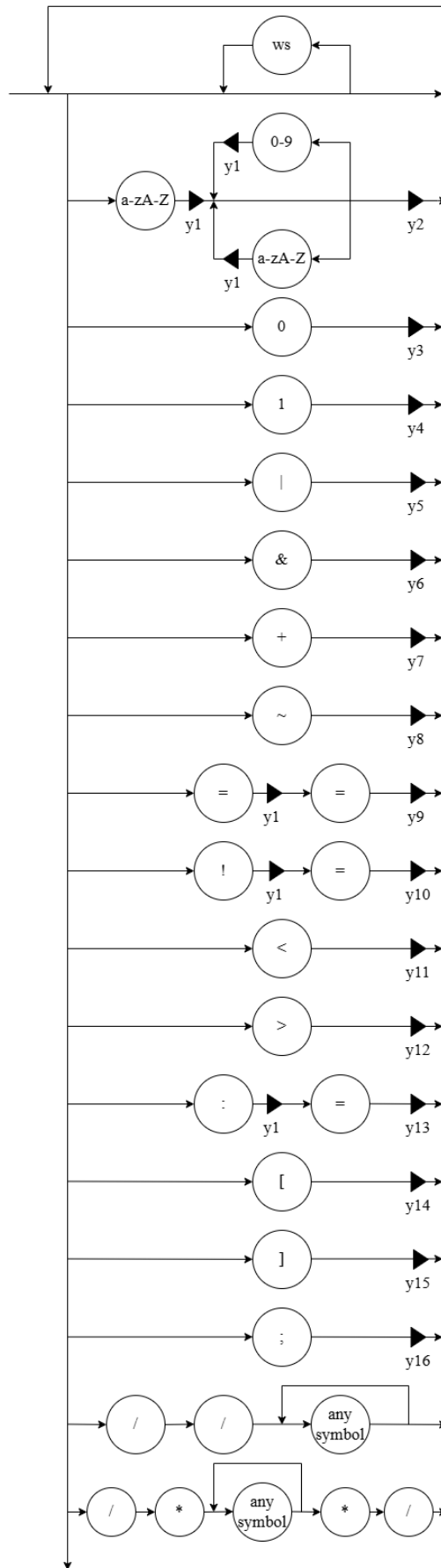


Рис. 3: Синтаксическая диаграмма

1.5 Обработка ошибок

Лексический анализатор обнаруживает следующие ошибки:

- недопустимые символы: $\exists c \notin \Sigma$;
- превышение максимальной длины идентификатора: $|id| > 16$;
- начало имени идентификатора с цифры;
- некорректные числовые значения: допускаются только константы 0 и 1.

Для каждой ошибки генерируется сообщение с описанием ошибки.

2 Особенности реализации

2.1 Класс TokenType

Класс TokenType представляет собой перечисление (Enum), определяющее возможные типы лексем в языке.

Код класса приведен в листинге 1.

Листинг 1 Класс TokenType

```
1 class TokenType(Enum):
2     IDENTIFIER = "Идентификатор"
3     CONSTANT = "Константа"
4     OPERATOR = "Логический оператор"
5     COMPARASION = "Оператор сравнения"
6     ASSIGN = "Оператор присваивания"
7     BRACKET = "Скобка"
8     SEMICOLON = "Разделитель (точка с запятой)"
9     ERROR = "Ошибка"
10    EOF = "Конец файла"
```

2.2 Класс Token

Класс Token представляет собой лексему (токен) – результат работы лексического анализатора.

Поля класса

- `type` – тип токена (значение из TokenType),
- `value` – строковое значение токена,
- `line` – номер строки в исходном коде,
- `column` – позиция в строке.

Метод `__str__` формирует строковое представление токена с информацией о нём для вывода.

Код класса приведен в листинге 2.

Листинг 2 Класс Token

```
1 class Token:
2     def __init__(self, type_, value, line, column):
3         self.type = type_
4         self.value = value
5         self.line = line
6         self.column = column
7
8     def __str__(self):
9         return f"Токен({self.type.value}, '{self.value}', строка {self.line}, позиция {self.column})"
```

2.3 Класс LexicalAnalyzer

Класс `LexicalAnalyzer` является основным классом лексического анализатора, выполняющим разбор исходного кода на лексемы (токены).

2.3.1 Конструктор класса

Конструктор инициализирует анализатор, определяет допустимые токены с использованием регулярных выражений.

Конструктор инициализирует следующие компоненты:

- `token_patterns` – список кортежей, связывающих тип токена с регулярным выражением для его распознавания:
 - `ASSIGN` – оператор присваивания `():=`,
 - `COMPARASION` – операторы сравнения: двухсимвольные `(==, !=)` и односимвольные `(<, >)`,
 - `OPERATOR` – логические операторы `(|, &, +, ~)`,
 - `BRACKET` – квадратные скобки,
 - `SEMICOLON` – разделитель `(;)`,
- `whitespace_pattern` – для пропуска пробелов;
- `identifier_pattern` – для обработки идентификаторов (латинские буквы, может содержать цифры);
- `alnum_sequence_pattern` – для обработки ошибок в последовательностях, содержащих буквы и цифры;
- `identifier` – словарь для хранения идентификаторов и их номеров;
- `max_identifier_len` – максимальная длина идентификатора (15).

Код конструктора приведен в листинге 3.

Листинг 3 Конструктор класса `LexicalAnalyzer`

```
1 def __init__(self):
2     self.token_patterns = [
3         (TokenType.ASSIGN, re.compile(r':=')),
4         (TokenType.COMPARASION, re.compile(r'==|!=')),
5         (TokenType.COMPARASION, re.compile(r'[<>]')),
6         (TokenType.OPERATOR, re.compile(r'[|&+~]')),
7         (TokenType.BRACKET, re.compile(r'[\[\]]')),
8         (TokenType.SEMICOLON, re.compile(r';')),
9     ]
10
11     self.whitespace_pattern = re.compile(r'[\t]+')
```

```

12
13     # Паттерн для идентификатора
14     self.identifier_pattern = re.compile(r'[a-zA-Z][a-zA-Z0-9_]*')
15
16     # Паттерн для последовательности из букв, цифр и подчеркиваний
17     self.alnum_sequence_pattern = re.compile(r'[a-zA-Z0-9_]+')
18
19     self.identifiers = {}
20     self.next_identifier_num = 1
21     self.max_identifier_len = 15

```

2.3.2 Методы класса

Метод `remove_comments`

Вход: `text` – исходный текст программы.

Выход: текст программы без комментариев.

Метод удаляет комментарии из исходного кода и возвращает очищенный от комментариев текст программы. Код метода приведен в листинге 4.

Листинг 4 Метод `remove_comments`

```

1 def remove_comments(self, text):
2     text=re.sub(r'//.*$', '', text, flags=re.MULTILINE)
3     text = re.sub(r'/\*.*?\*/', '', text, flags=re.DOTALL)
4     return text

```

В методе используются регулярные выражения для удаления однострочных и многострочных комментариев.

1. `r'//.*$'` – удаляет однострочные комментарии, начинающиеся с `'//'`:
 - `//` – символы начала комментария,
 - `.*` – любое количество любых символов, кроме переноса строки,
 - `$` – конец строки,
 - флаг `re.MULTILINE` позволяет обрабатывать каждую строку отдельно.
2. `r'/\*.*?\*/'` – удаляет многострочные комментарии вида `/* ... */`:
 - `/\*` – экранированный символ `/*`,
 - `.*?` – любое количество любых символов,
 - `\*/` – экранированный символ `*/`,
 - флаг `re.DOTALL` позволяет символу `.` соответствовать также символам новой строки.

Метод `analyze`

Вход: `text` – исходный текст программы после удаления комментариев.

Выход: `tokens` – список объектов `Token`, представляющих лексемы; `errors` – список строк с описанием ошибок, обнаруженных после лексического анализа.

Метод выполняет лексический анализ исходного кода, разбивая его на последовательность лексем.

Листинг 5 Метод `analyze`

```
1 def analyze(self, text):
2     tokens = []
3     errors = []
4
5     # Удаляем комментарии
6     text = self.remove_comments(text)
7
8     pos = 0
9     line = 1
10    line_start = 0
11    length = len(text)
12
13    while pos < length:
14        # Пропускаем пробелы и табуляции
15        whitespace_match = self.whitespace_pattern.match(text,
16            pos)
17        if whitespace_match:
18            pos = whitespace_match.end()
19            continue
20
21        # Проверяем на перевод строки
22        if text[pos] == '\n':
23            line += 1
24            line_start = pos + 1
25            pos += 1
26            continue
27
28        column = pos - line_start + 1
29        matched = False
30
31        # Сначала проверяем операторы и разделители
32        for token_type, pattern in self.token_patterns:
33            match = pattern.match(text, pos)
34            if match:
35                lexeme = match.group()
36                tokens.append(Token(token_type, lexeme, line,
37                    column))
38                pos = match.end()
39                matched = True
40                break
41
42        if matched:
```

```

41         continue
42
43     # Проверяем, начинается ли последовательность с цифры
44     if pos < length and text[pos].isdigit():
45         # Захватываем всю последовательность из букв, цифр и
46           подчеркиваний
47         sequence_match = self.alnum_sequence_pattern.match(
48             text, pos)
49         if sequence_match:
50             lexeme = sequence_match.group()
51
52             # Если это просто цифра 0 или 1 и это вся последо-
53               вательность
54             if lexeme in {'0', '1'} and len(lexeme) == 1:
55                 # Проверяем, не является ли это частью больше-
56                   го числа
57                 tokens.append(Token(TokenType.CONSTANT,
58                     lexeme, line, column))
59                 pos = sequence_match.end()
60             else:
61                 # Это ошибка: идентификатор не может начинать-
62                   ся с цифры
63                 errors.append(
64                     f"Ошибка: идентификатор не может начинать-
65                       ся с цифры '{lexeme}'")
66                 tokens.append(Token(TokenType.ERROR, lexeme,
67                     line, column))
68                 pos = sequence_match.end()
69                 continue
70
71     # Проверяем на идентификатор
72     identifier_match = self.identifier_pattern.match(text,
73         pos)
74     if identifier_match:
75         lexeme = identifier_match.group()
76
77         if len(lexeme) > self.max_identifier_len:
78             errors.append(
79                 f"Ошибка: длина идентификатора '{lexeme}' пре-
80                   вышает {self.max_identifier_len} символов"
81             )
82             tokens.append(Token(TokenType.ERROR, lexeme, line
83                 , column))
84         else:
85             if lexeme not in self.identifiers:
86                 self.identifiers[lexeme] = self.
87                     next_identifier_num
88                 self.next_identifier_num += 1
89             tokens.append(Token(TokenType.IDENTIFIER, lexeme,
90                 line, column))
91
92     pos = identifier_match.end()

```

```
79         continue
80
81     # Если ничего не подошло - недопустимый символ
82     errors.append(f"Ошибка: недопустимый символ '{text[pos]}'")
83     tokens.append(Token(TokenType.ERROR, text[pos], line,
84                        column))
85     pos += 1
86
87 last_column = column if 'column' in locals() else 1
88 tokens.append(Token(TokenType.EOF, 'EOF', line, last_column))
89
90 return tokens, errors
```

3 Результаты работы

Программа №1 содержит круглые скобки, не допускаемые анализатором. Остальные символы и подцепочки соответствуют языку.

```
// программа с круглыми скобками
x := 1;
y := 0;
result := x & y | (x + y);
final := [x == 1] & (y != 0);
```

Далее представлен вывод для программы №1.

Найденные ошибки:

```
Ошибка: недопустимый символ '('
Ошибка: недопустимый символ ')'
Ошибка: недопустимый символ '('
Ошибка: недопустимый символ ')'
```

ТАБЛИЦА ЛЕКЕМ

Тип	Значение	Номер идентификатора
Идентификатор	x	x:1
Оператор присваивания	:=	
Константа	1	
Разделитель (точка с запятой)	;	
Идентификатор	y	y:2
Оператор присваивания	:=	
Константа	0	
Разделитель (точка с запятой)	;	
Идентификатор	result	result:3
Оператор присваивания	:=	
Идентификатор	x	x:1
Логический оператор	&	
Идентификатор	y	y:2
Логический оператор		
Идентификатор	x	x:1
Логический оператор	+	
Идентификатор	y	y:2
Разделитель (точка с запятой)	;	
Идентификатор	final	final:4
Оператор присваивания	:=	

Скобка	[	
Идентификатор	x	x:1
Оператор сравнения	==	
Константа	1	
Скобка	]	
Логический оператор	&	
Идентификатор	y	y:2
Оператор сравнения	!=	
Константа	0	
Разделитель (точка с запятой)	;	

=====

Программа №2 содержит некорректные имена идентификаторов и недопустимые символы.

```
/* ffffffff
fffff */
2var := 1;
very_long_identififer_example := 0;
x := 2; // полный...
a := b @ c;
correct := 1;
```

Далее приведен вывод для программы №2.

Найденные ошибки:

Ошибка: идентификатор не может начинаться с цифры '2var'
 Ошибка: длина идентификатора 'very_long_identififer_example'
 превышает 15 символов
 Ошибка: идентификатор не может начинаться с цифры '2'
 Ошибка: недопустимый символ '@'

=====

ТАБЛИЦА ЛЕКСЕМ

=====

Тип	Значение	Номер идентификатора
Оператор присваивания	:=	
Константа	1	
Разделитель (точка с запятой)	;	
Оператор присваивания	:=	
Константа	0	
Разделитель (точка с запятой)	;	
Идентификатор	x	x:1

```

Оператор присваивания      :=
Разделитель (точка с запятой) ;
Идентификатор               a           a:2
Оператор присваивания      :=
Идентификатор               b           b:3
Идентификатор               c           c:4
Разделитель (точка с запятой) ;
Идентификатор               correct      correct:5
Оператор присваивания      :=
Константа                   1
Разделитель (точка с запятой) ;
=====

```

Программа №3 содержит некорректные имена идентификаторов и содержит недопустимые символы.

```

x0 := 1;
1y ::= 0;
x !=== x | y;
z := [x & уу ] ;
meow = 108

```

Ниже приведен вывод для программы №3.

Найденные ошибки:

- Ошибка: идентификатор не может начинаться с цифры '1y'
- Ошибка: недопустимый символ ':'
- Ошибка: недопустимый символ '='
- Ошибка: недопустимый символ '='
- Ошибка: недопустимый символ '='
- Ошибка: идентификатор не может начинаться с цифры '108'

ТАБЛИЦА ЛЕКЕМ

Тип	Значение	Номер идентификатора
Идентификатор	x0	x0:1
Оператор присваивания	:=	
Константа	1	
Разделитель (точка с запятой)	;	
Оператор присваивания	:=	
Константа	0	
Разделитель (точка с запятой)	;	

Идентификатор	x	x:2
Оператор сравнения	!=	
Оператор сравнения	==	
Идентификатор	x	x:2
Логический оператор		
Идентификатор	y	y:3
Разделитель (точка с запятой)	;	
Идентификатор	z	z:4
Оператор присваивания	:=	
Скобка	[	
Идентификатор	x	x:2
Логический оператор	&	
Идентификатор	yy	yy:5
Скобка	]	
Разделитель (точка с запятой)	;	
Идентификатор	meow	meow:6

=====

Заключение

В результате выполнения лабораторной работы реализован лексический анализатор для входного языка, содержащего только латиницу и логические выражения, разделенные символом ; (точка с запятой). Язык был представлен в виде синтаксической диаграмме, а также был написан код на языке Python, обеспечивающий работу синтаксического анализатора.

Достоинства реализации

1. Программа обеспечивает как ручной ввод программы, так и чтение файлов, переданных в качестве аргумента командной строки.
2. Проверка токенов осуществляется с использованием регулярных выражений, что существенно ускоряет процесс лексического анализа.

Недостатки реализации

1. Для каждой позиции в тексте, переданном анализатору, выполняется последовательная проверка всех шаблонов, пока не будет встречено совпадение. При наличии ошибки в тексте программы проверяются все шаблоны, что неэффективно для больших программ.
2. Несмотря на использование регулярных выражений для распознавания, дополнительная валидация все равно выполняется вручную.

Масштабирование программы

Для масштабирования программы можно расширить набор типов токенов (например, добавить служебные слова) или расширить множества уже имеющихся типов лексем (например, добавить новые логические операторы, нестрогие операторы сравнения).

Список литературы

- [1] Секция «Телематика» ВШТИИ. – [Электронный ресурс] (дата обращения: 05.03.2026).